

GH Archive Data Profiling, Cleaning, and Ingestion Report

1. Introduction

This report documents the ingestion, profiling, and cleaning workflow for the GH Archive data used in the OSS Pulse project. The objective of this phase is to convert raw GitHub public event data into a structured dataset that can support downstream Spark analytics on AI-related open-source repositories.

2. Data Source Description

- ****Data source****: GH Archive
- ****Owner / access****: Public GitHub activity data archived by GH Archive
- ****Original format****: Hourly `JSON.gz` files
- ****Collection scope for this submission****: Replace this sentence with the exact time window you processed
- ****Relevant event types****: `WatchEvent`, `ForkEvent`, `PushEvent`, `PullRequestEvent`, `IssuesEvent`

Core Fields

- `id`
- `type`
- `created\_at`
- `actor`
- `repo`
- `payload`
- `public`
- `org`

3. Ingestion Workflow

Describe how the raw GH Archive files were loaded into Spark.

- Script used: `ingestion/ingest\_gharchive.py`
- Spark API used: DataFrame API
- Input format: `JSON.gz`
- Output format: raw Parquet

Ingestion Notes

- State the input path you used
- State whether you applied an explicit schema
- Mention where the raw Parquet output was written

4. Data Profiling

Describe the exploratory profiling you ran on the raw dataset.

- Script used: `profiling/profile\_gharchive.py`
- Total number of records: `TODO`
- Total number of columns: `TODO`
- Distinct repositories: `TODO`
- Distinct actors: `TODO`
- Event timestamp range: `TODO`

4.1 Event Type Distribution

Summarize the distribution of GitHub event types and include a small table or chart.

4.2 Missing Values and Data Quality

Summarize important null patterns, for example:

- missing `actor.login`
- missing `repo.name`
- missing `payload.action`
- malformed or null timestamps

4.3 Observed Data Issues

Document any issues you found, such as:

- nested JSON complexity in `payload`
- inconsistent payload shape across event types
- duplicated event IDs
- noisy repositories outside the final analysis scope

5. Data Cleaning

Describe the transformations applied in `cleaning/clean\_gharchive.py`.

Cleaning Steps

1. Filter to the five core event types used by the project.
2. Flatten nested fields from `actor`, `repo`, `org`, and `payload`.
3. Convert `created\_at` into `event\_ts` and `event\_date`.
4. Normalize repository and actor names to lowercase.
5. Drop records with missing repository names or invalid timestamps.
6. Remove duplicate records using `event\_id`.
7. Optionally filter to a curated repository seed list.

Cleaned Output Schema

List the main cleaned columns here:

- `event\_id`
- `event\_type`
- `event\_ts`
- `event\_date`
- `actor\_id`
- `actor\_login`
- `repo\_id`
- `repo\_name`
- `payload\_action`
- `issue\_number`
- `pr\_number`
- `pr\_merged`
- `commit\_count`

6. Dataset Snippet

Insert a small snippet from the raw dataset and a small snippet from the cleaned dataset. Do not include the full dataset.

Raw Data Sample

Paste or screenshot `output/samples/raw\_gharchive\_sample.csv`

Cleaned Data Sample

Paste or screenshot `output/samples/clean\_gharchive\_sample.csv`

7. Output for the Next Project Stage

Explain how the cleaned GH Archive dataset will support the next stage of the project.

Suggested points:

- daily repository metrics
- trend analysis for stars, forks, issues, and pull requests
- future join with Hugging Face repository-linked metadata
- lightweight frontend views for repository detail and comparison pages

8. Appendix

Include command examples if helpful.

```
```bash
python ingestion/ingest_gharchive.py --input "data/source/2025-01-01-*.json.gz"
python profiling/profile_gharchive.py --input data/raw/gharchive --input-format
parquet
python cleaning/clean_gharchive.py --input data/raw/gharchive --input-format par
quet
```
```